Assignment 1 - Streamlit app

Start Assignment

Due Tuesday by 11:59pm **Points** 10 **Submitting** a text entry box, a website url, or a file upload

Homework Assignment: Building a Sentiment Analysis Web App with Streamlit

Objective: The goal of this assignment is to build a complete, end-to-end machine learning application. You will train a sentiment analysis model on movie review data, save it, and then build an interactive web app with Streamlit that allows a user to input any text and see the predicted sentiment. I have included sufficient hints and comments to assist you with completing this assignment easily. Feel free to email if you face any confusion.

Due Date: Tuesday, 30th June. 11.59 pm MT

Part 1: Data Preparation and Model Training

In this part, you will prepare the data, train a Naive Bayes classifier, and save the trained model pipeline.

Step 1: Get the Data

We will use the Large Movie Review Dataset (IMDB). For simplicity, you can use a pre-processed version available on Kaggle.

- **Dataset:** IMDB Dataset of 50K Movie Reviews
- Download the `IMDB Dataset.csv` file from the link above and place it in your project folder.

Step 2: Create a Training Script

Create a Python script named train_model.py. This script will be responsible for loading the data, training the model, and saving it.

Step 3: Load and Preprocess the Data

- Use `pandas` to load the `IMDB Dataset.csv` file.
- The dataset has two columns: `review` and `sentiment`. The sentiment is already conveniently labeled as `positive` or `negative`.
- You will need to split your data into features (the `review` text) and labels (the `sentiment`). Let's call them `X` and `y`.

Step 4: Train the Model

For this task, a combination of TfidfVectorizer and MultinomialNB (Naive Bayes) is a strong and classic baseline. To make the model easy to use in, package them together in a Pipeline.

- Import `Pipeline` from `sklearn.pipeline`, `TfidfVectorizer` from `sklearn.feature_extraction.text`, and `MultinomialNB` from `sklearn.naive_bayes`.
- Create a pipeline that first transforms the text data using `TfidfVectorizer` and then feeds it to the `MultinomialNB` classifier.
- Train the pipeline on your entire dataset (`X` and `y`). No need to create a train-test split for this assignment

Step 5: Save the Model Pipeline

- Once the model is trained, you need to save it to a file so your Streamlit app can use it later.
- Use the `joblib` library to dump your trained `Pipeline` object into a file named `sentiment_model.pkl`.

Your `train_model.py` script should only be run once to generate the `sentiment_model.pkl` file.

Part 2: Building the Streamlit Application

Now for the fun part! You will create a web interface for your model.

Step 1: Create the App Script

Create a new Python script named app.py.

Step 2: Set up the Basic App Layout

- Import `streamlit` and `joblib`.
- Give your application a title, for example: `Movie Review Sentiment Analyzer`.
- Write a short description of what the app does.

Step 3: Load the Saved Model

- Write a function to load `sentiment_model.pkl` using `joblib.load()`.
- **Crucially, use the `@st.cache_data` decorator on this function (review Lab 1.5).** This ensures the model is loaded only once when the app starts, which is essential for performance.

Step 4: Create the User Input Interface

- Use `st.text_area()` to create a text box where the user can type or paste a movie review. Give it a descriptive label like "Enter a movie review to analyze:".
- Add a button with `st.button()` labeled "Analyze".

Step 5: Make Predictions and Display Results

- Write an `if` block that checks if the "Analyze" button has been pressed.
- Inside the `if` block:
  1. Get the text from the text area.
  2. Make sure the user has entered some text before trying to make a prediction.
  3. Use your loaded model pipeline's `.predict()` method on the user's text. Note that the pipeline expects a list or array of documents, so you'll need to pass the input text inside a list (e.g., `[user_text]`).
  4. The output will be the predicted sentiment (`'positive'` or `'negative'`).
  5. Display the result to the user in a clear way. Use `st.subheader()` or `st.write()` to show the prediction.
  6. Display the prediction probability using the model's `.predict_proba()` method.
     - **Pro-tip:** Make the output more engaging! If the sentiment is positive, you could write "Predicted Sentiment: Positive 👍" and if it's negative, "Predicted Sentiment: Negative 👎".

Step 6: Run Your App

Open your terminal in the project directory and run:

```
streamlit run app.py
```

Test your app with different reviews to see if it works as expected.

Submission Guidelines

To receive full grade, you must push all the files on a GitHub repository (make sure it's public or add my email if you want to keep it private: nmashinchi@gmail.com).

You only have to add the GitHub URL when submitting the assignment. You can work in pairs; however, make sure to submit your work individually in separate accounts and repositories.

Ensure to include the following files:

1. `train_model.py` (your model training script)
2. `app.py` (your Streamlit application script)
3. `sentiment_model.pkl` (the saved model file)
4. A `requirements.txt` file listing the libraries needed to run your project (e.g., `streamlit`, `pandas`, `scikit-learn`, `joblib`).
5. A README file with a simple paragraph on how to clone and run your app locally. Preferably, write it in bullet points.

Bonus Points (Optional)

For extra credit, you can add one or more of the following features:

- Display the prediction probability using the model's `.predict_proba()` method.
- Style the output with color. For example, show positive predictions in green and negative predictions in red. (Hint: `st.success()` and `st.error()`)